DATA 612 · Statistical Programming in R · Week 7

7.2 tidyr: pivot_wider(), separate(), unite()

Widen a key/value pair, split a combined column, or paste columns back together.

1. `pivot_wider()` is the inverse

`table2` is long: a `type` column (`"cases"` or `"population"`) and a `count` column. That is tidy in a different way. For a rate you want cases and population **side by side**.

```
library(tidyverse)

table2 |>
  pivot_wider(names_from = type, values_from = count)
```

`names_from` is the column whose **values** become new names. `values_from` is the column that fills the cells. The other columns (`country`, `year`) stay as keys.

`pivot_longer()` then `pivot_wider()` (with matching arguments) should return the table you started with.

---

2. Duplicate keys fail

Each key must point to **one** cell. Two ages for the same person cannot both occupy an `age` column.

```
people <- tribble(
  ~name,             ~key,    ~value,
  "Phillip Woods",   "age",       45,
  "Phillip Woods",   "height",   186,
  "Phillip Woods",   "age",       50,
  "Jessica Cordero", "age",       37,
  "Jessica Cordero", "height",   156
)

people |>
  pivot_wider(names_from = key, values_from = value)
```

The error is that the rows are not uniquely identified. Add a row id (`mutate(id = row_number())`), or decide which age to keep, before you widen.

---

3. `separate()` splits one column

`table3` stores `rate` as `"745/19987071"`. Two values, one cell — not tidy.

```
table3 |>
  separate(rate, into = c("cases", "population"), sep = "/", convert = TRUE)
```

`into` names the new columns. `sep` can be a string or a position. `convert = TRUE` runs `type.convert()` so the pieces become numbers when they look like numbers.

Too many pieces: `extra = "merge"` or `"drop"`. Too few: `fill = "right"` or `"left"`.

---

4. `unite()` pastes columns

`table5` splits the year into `century` and `year`. Paste them back:

```
table5 |>
  unite(new, century, year, sep = "")
```

`sep = ""` glues with nothing in between. Default `sep` is `"_"`. `remove = FALSE` keeps the old columns.

---

5. Tidy, then join

`table4a` is cases; `table4b` is population. Lengthen both, then join on the keys. `left_join()` keeps every row of the left table and adds matching columns from the right. Note 8.1 is the full join story.

```
tidy4a <- table4a |>
  pivot_longer(cols = -country, names_to = "year", values_to = "cases")

tidy4b <- table4b |>
  pivot_longer(cols = -country, names_to = "year", values_to = "population")

left_join(tidy4a, tidy4b, by = c("country", "year"))
```

That result is `table1` (year may still be character).

---

6. Practice

Put course CSV files in a `data/` folder of your project.

1. `flowers1.csv` uses `;` and a comma as the decimal mark. Load it with `read_delim("data/flowers1.csv", delim = ";")`. If `Value` is character, set `locale = locale(decimal_mark = ",")`. Then `pivot_wider()` so `Flowers` and `Intensity` are columns.
2. Load `flowers2.csv`. `separate()` the `Flowers/Intensity` column. Then `unite()` those two columns again with a comma.

3. In `nycflights13::flights`, `unite()` `month`, `day`, `hour`, and `minute` into `sd_time`.